

UNIT-5

BAKTRACKING

Many problems are difficult to solve algorithmically. Backtracking makes it possible to solve at least some large instances of difficult combinatorial problems.

Suppose we have to make a series of decisions among various choices, where

- We don't have enough information to know what to choose
- Each decision leads to a new set of choices.
- Some sequence of choices (more than one choices) may be a solution to your problem.

Applications of Backtracking:

- N Queens Problem
- Sum of subsets problem
- Graph coloring
- Hamiltonian cycles.

Terminology:

Problem state is each node in the depth-first search tree

State space is the set of all paths from root node to other nodes

Solution states are the problem states s for which the path from the root node to s

Answer states are that solution states s for which the path from root node to s defines a tuple that is a member of the set of solutions

State space tree is the tree organization of the solution space

Live node is a generated node for which all of the children have not been generated yet.

E-node is a live node whose children are currently being generated or explored

Dead node is a generated node that is not to be expanded any further

N-Queens Problem:

A classic combinational problem is to place n queens on a $n \times n$ chess board so that no two attack, i.e. no two queens are on the same row, column or diagonal.

If we take $n=4$ then the problem is called 4 queens problem.

If we take $n=8$ then the problem is called as 8 queens problem.

4-Queens problem:

Consider a 4*4 chessboard. Let there are 4 queens. The objective is place there 4 queens on 4*4 chessboard in such a way that no two queens should be placed in the same row, same column or diagonal position.

The explicit constraints are 4 queens are to be placed on 4*4 chessboards in 44 ways.

The implicit constraints are no two queens are in the same row column or diagonal.

Let $\{x_1, x_2, x_3, x_4\}$ be the solution vector where x_i column on which the queen i is placed.

First queen is placed in first row and first column.

1			

(a)

The second queen should not be in first row and second column. It should be placed in second row and in second, third or fourth column. If we place in second column, both will be in same diagonal, so place it in third column.

1			
•	•		

(b)

1			
•	•	2	
•	•	•	•

(c)

We are unable to place queen 3 in third row, so go back to queen 2 and place it somewhere else

1			
			2

(d)

1			
			2
	3		

(e)

Now the fourth queen should be placed in 4th row and 3rd column but there will be a diagonal attack from queen 3. So go back, remove queen 3 and place it in the next column. But it is not possible, so move back to queen 2 and remove it to next column but it is not possible. So go back to

	1		

	1		
			2

queen 1 and move it to next column.

(f)

	1		
			2
3			

(h)

(g)

1			
			2
3			
		4	

(i)

Fig: Example of Backtrack solution to the 4-queens problem

Hence the solution of to 4-queens's problem is $x_1=2$, $x_2=4$, $x_3=1$, $x_4=3$, i.e first queen is placed in 2nd column, second queen is placed in 4th column and third queen is placed in first column and fourth queen is placed in third column.

1									
x ₁ =1					x ₁ =2				
2					3				
x ₂ =2		x ₂ =3		x ₂ =4		x ₂ =1		x ₂ =4	
4		5		6		7		9	
R		x ₃ =2		x ₃ =4		x ₃ =2		x ₃ =3	
		10		11		12		13	
				x ₄ =3		R			
								15	
								x ₄ =3	

8-queens problem

A classic combinatorial problem is to place 8 queens on a 8*8 chess board so that no two attack, i.e no two queens are to the same row, column or diagonal.

Now, we will solve 8 queens problem by using similar procedure adapted for 4 queens problem. The algorithm of 8 queens problem can be obtained by placing $n=8$, in N queens algorithm.

If two queens are placed at positions (i,j) and (k,l). They are on the same diagonal only if
 $i-j=k-l$ (1) or
 $i+j=k+l$ (2).

From (1) and (2) implies $j-l=i-k$ and $j-l=k-i$

Two queens lie on the same diagonal iff $|j-l|=|i-k|$

The solution of 8 queens problem can be obtained similar to the solution of 4 queens.

problem. $X_1=3, X_2=6, X_3=2, X_4=7, X_5=1, X_6=4, X_7=8, X_8=5$,
The solution can be shown as

		1					
					2		
	3						
						4	
5							
			6				
							7
				8			

2) Sum of Subsets Problem

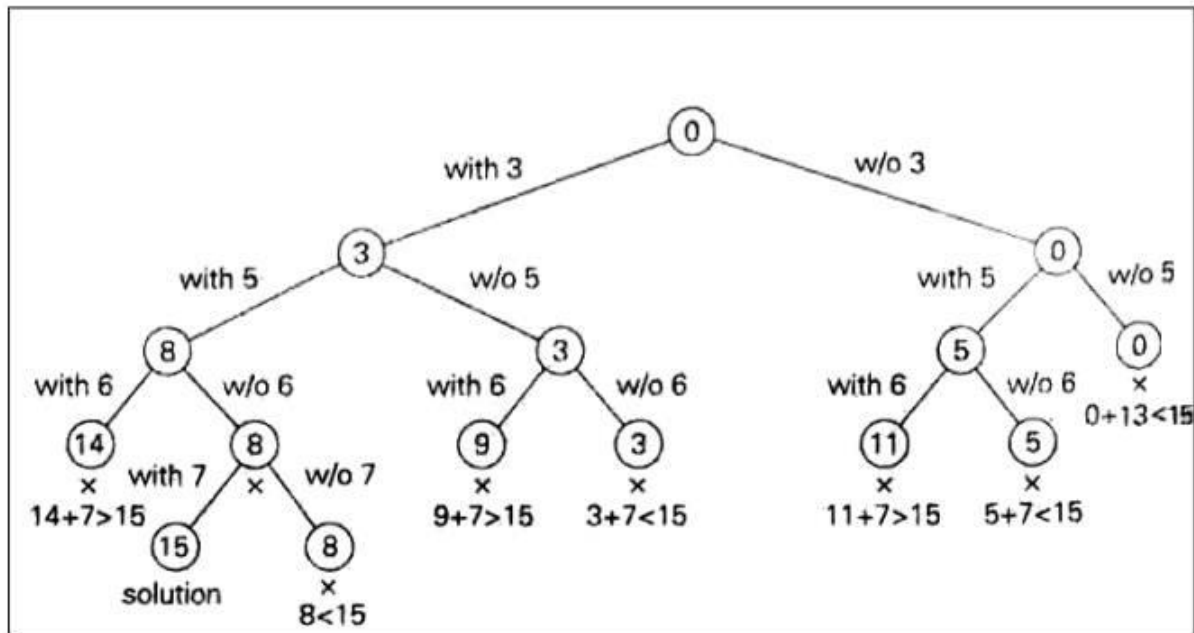
Subset sum problem is the problem of finding a subset such that the sum of elements equal a given number. The backtracking approach generates all permutations in the worst case but in general, performs better than the recursive approach towards subset sum problem.

A subset A of n positive integers and a value sum(d) is given, find whether or not there exists any subset of the given set, the sum of whose elements is equal to the given value of sum.

Steps:

1. Start with an empty set
2. Add the next element from the list to the set
3. If the subset is having sum M, then stop with that subset as solution.
4. If the subset is not feasible or if we have reached the end of the set, then backtrack through the subset until we find the most suitable value.
5. If the subset is feasible (sum of subset $< d$) then go to step 2.
6. If we have visited all the elements without finding a suitable subset and if no backtracking is possible then stop without solution.

Example: $S = \{3,5,6,7\}$ and $d = 15$, Find the sum of subsets by using backtracking



Solution {3,5,7}

3) Hamiltonian Circuits Problem

A Hamiltonian circuit or tour of a graph is a path that starts at a given vertex, visits each vertex in the graph exactly once, and ends at the starting vertex.

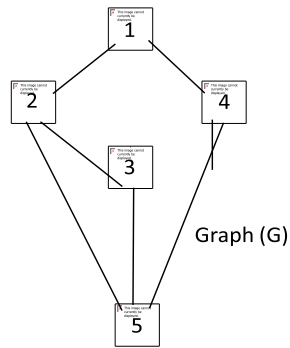
We use the Depth-First Search algorithm to traverse the graph until all the vertices have been visited.

We traverse the graph starting from a vertex (arbitrary vertex chosen as starting vertex) and at any point during the traversal we get stuck (i.e., all the neighbor vertices have been visited), we backtrack to find other paths (i.e., to visit another unvisited vertex).

If we successfully reach back to the starting vertex after visiting all the nodes, it means the graph has a Hamiltonian cycle otherwise not.

We mark each vertex as visited so that we don't traverse them more than once.

Example: Find the Hamiltonian circuits of a given graph using Backtracking.

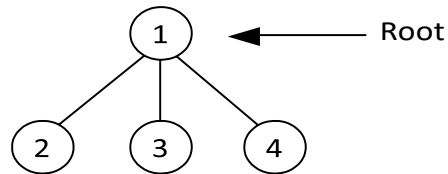


Initially we start out search with vertex '1' the vertex '1' becomes the root of our implicit tree.



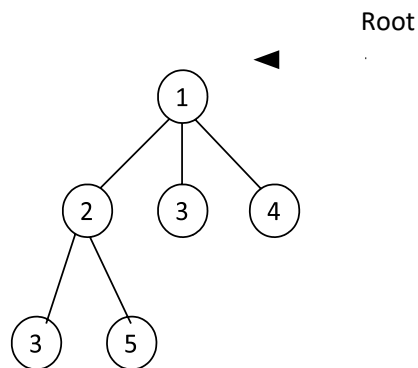
(a)

Next we choose vertex '2' adjacent to '1', as it comes first in numerical order (2, 3, 4).



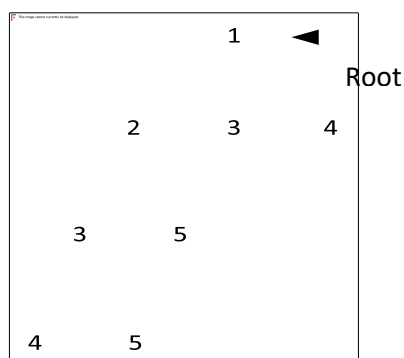
(b)

Next vertex '3' is selected which is adjacent to '2' and which comes first in numerical order (3,5).

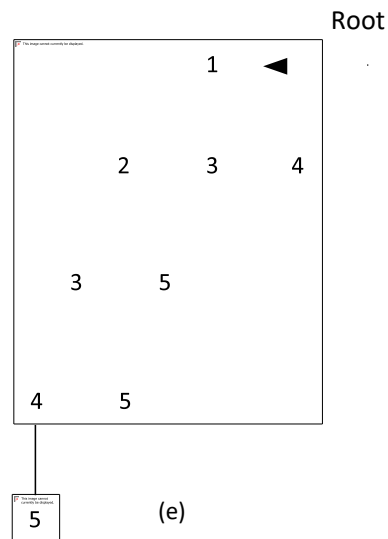


(c)

Next we select vertex '4' adjacent to '3' which comes first in numerical order (4, 5).



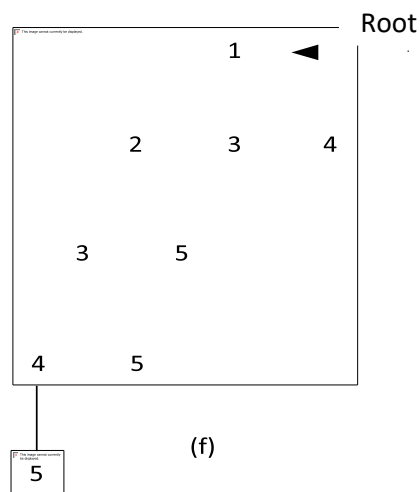
Next vertex '5' is selected. If we choose vertex '1' then we do not get the Hamiltonian cycle.



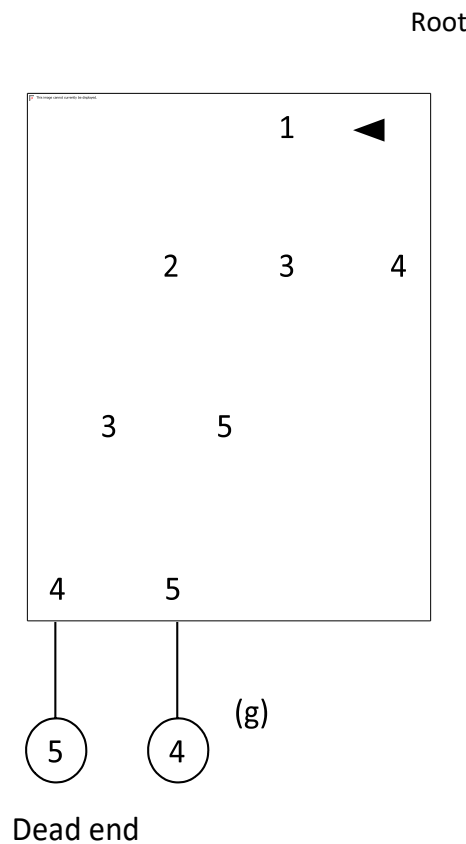
The vertex adjacent to 5 is 2, 3, 4 but they are already visited. Thus, we get the dead end. So, we backtrack one step and remove the vertex '5' from our partial solution.

The vertex adjacent to '4' are 5,3,1 from which vertex '5' has already been checked and we are left with vertex '1' but by choosing vertex '1' we do not get the Hamiltonian cycle. So, we again backtrack one step.

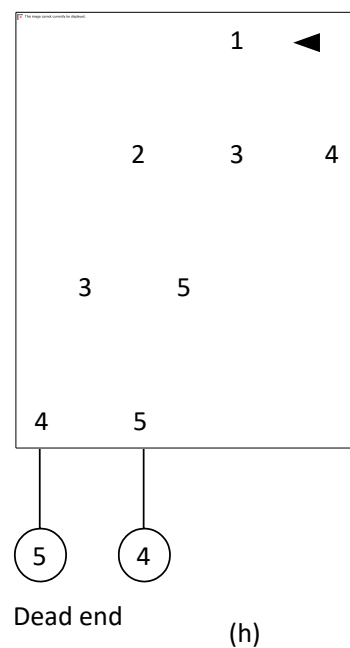
Hence we select the vertex '5' adjacent to '3'.



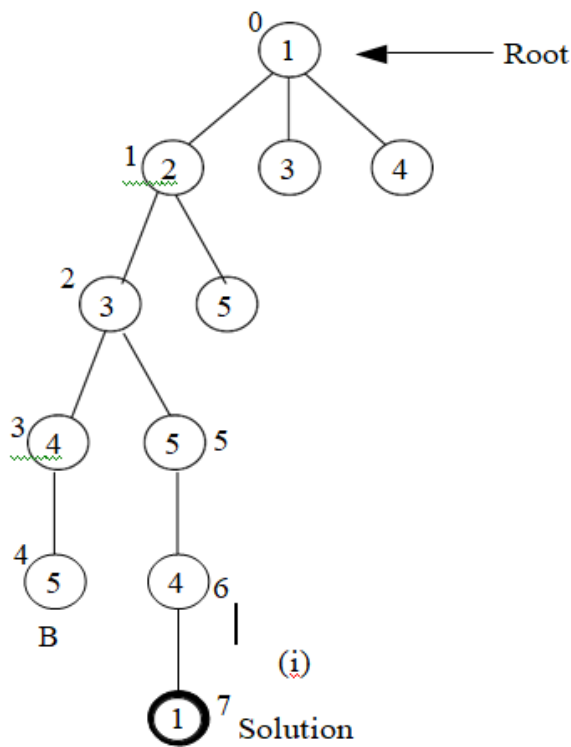
The vertex adjacent to '5' are (2,3,4) so vertex 4 is selected.



The vertex adjacent to '4' are (1, 3, 5) so vertex '1' is selected. Hence we get the Hamiltonian cycle as all the vertex other than the start vertex '1' is visited only once, 1- 2- 3- 5- 4- 1.



The final implicit tree for the Hamiltonian circuit is shown below. The number above each node indicates the order in which these nodes are visited.



Algorithm:

```

1  Algorithm Hamiltonian( $k$ )
2  // This algorithm uses the recursive formulation of
3  // backtracking to find all the Hamiltonian cycles
4  // of a graph. The graph is stored as an adjacency
5  // matrix  $G[1 : n, 1 : n]$ . All cycles begin at node 1.
6  {
7      repeat
8      { // Generate values for  $x[k]$ .
9          NextValue( $k$ ); // Assign a legal next value to  $x[k]$ .
10         if ( $x[k] = 0$ ) then return;
11         if ( $k = n$ ) then write ( $x[1 : n]$ );
12         else Hamiltonian( $k + 1$ );
13     } until (false);
14 }

```

Branch and Bound

General method:

Branch and Bound is another method to systematically search a solution space. Just like backtracking, we will use bounding functions to avoid generating subtrees that do not contain an answer node.

Branch and Bound differs from backtracking in two important points:

- It has a branching function, which can be a depth first search, breadth first search or based on bounding function.
- It has a bounding function, which goes far beyond the feasibility test as a mean to prune efficiently the search tree.

Branch and Bound refers to all state space search methods in which all children of the E-node are generated before any other live node becomes the E-node.

- Live node is a node that has been generated but whose children have not yet been generated.
- E-node is a live node whose children are currently being explored. In other words, an E-node is a node currently being expanded.
- Dead node is a generated node that is not to be expanded or explored any further. All children of a dead node have already been expanded.
- Branch-and-bound refers to all state space search methods in which all children of an E-node are generated before any other live node can become the E-node.

Assignment Problem

Assigning n people to n jobs so that the total cost of the assignment is as small as possible. Select one element in each row of the matrix so that no two selected elements are in the same column and their sum is the smallest possible.

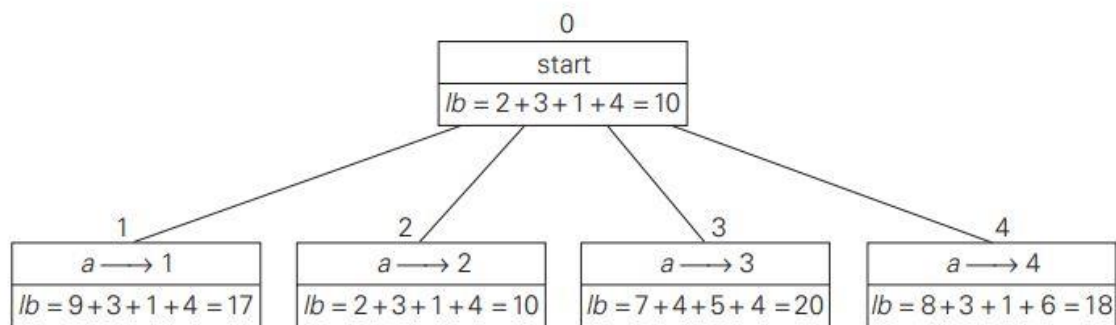
Let there be N workers and N jobs. Any worker can be assigned to perform any job, incurring some cost that may vary depending on the job assignment. It is required to perform all jobs by assigning exactly one worker to each job and exactly one job to each agent in such a way that the total cost of the assignment is minimized.

$$C = \begin{matrix} & \begin{matrix} \text{job 1} & \text{job 2} & \text{job 3} & \text{job 4} \end{matrix} \\ \begin{bmatrix} 9 & 2 & 7 & 8 \\ 6 & 4 & 3 & 7 \\ 5 & 8 & 1 & 8 \\ 7 & 6 & 9 & 4 \end{bmatrix} & \begin{matrix} \text{person } a \\ \text{person } b \\ \text{person } c \\ \text{person } d \end{matrix} \end{matrix}$$

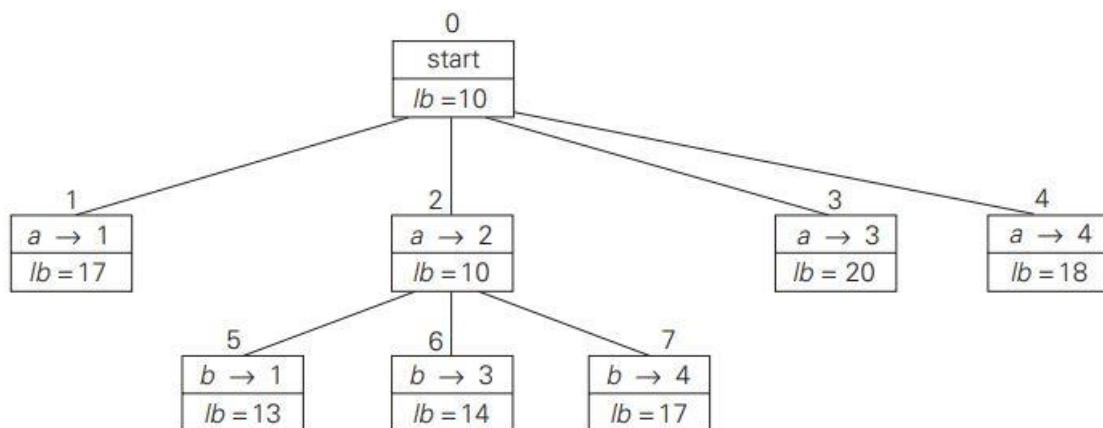
Lb= the sum of the smallest elements in each of the matrix's rows.

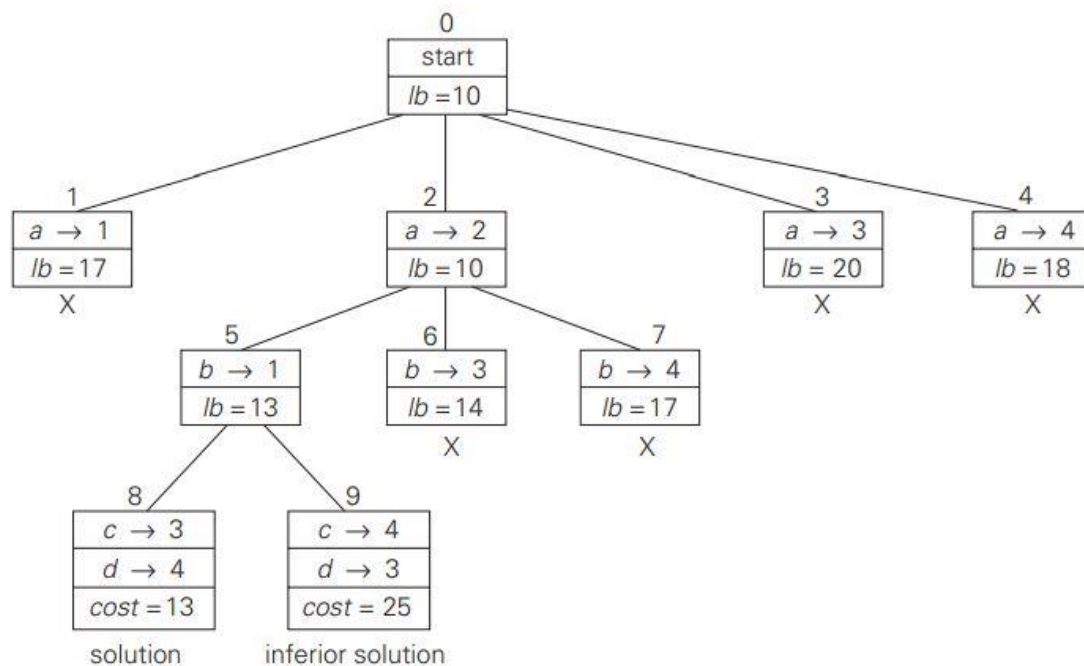
For the instance here, this sum is $2 + 3 + 1 + 4 = 10$.

The lower-bound value for the root, denoted lb , is 10. The nodes on the first level of the tree correspond to selections of an element in the first row of the matrix, i.e., a job for person a



The most promising of them is node 2 because it has the smallest lower bound value. We branch out from that node first by considering the three different ways of selecting an element from the second row and not in the second column the three different jobs that can be assigned to person b .





Traveling Sale Person Problem:

Example: Find the LC branch and bound solution for the traveling sale person problem whose cost matrix is as follows

$$\begin{pmatrix} \infty & 20 & 30 & 10 & 11 \\ 15 & \infty & 16 & 4 & 2 \\ 3 & 5 & \infty & 2 & 4 \\ 19 & 6 & 18 & \infty & 3 \\ 16 & 4 & 7 & 16 & \infty \end{pmatrix}$$

Step 1: Find the reduced cost matrix.

Apply row reduction method:

Deduct 10 (which is the minimum) from all values in the 1st row.

Deduct 2 (which is the minimum) from all values in the 2nd row.

Deduct 2 (which is the minimum) from all values in the 3rd row.

Deduct 3 (which is the minimum) from all values in the 4th row.

Deduct 4 (which is the minimum) from all values in the 5th row.

The resulting row wise reduced cost matrix =

$$\begin{pmatrix} \infty & 10 & 20 & 0 & 1 \\ 13 & \infty & 14 & 2 & 0 \\ 1 & 3 & \infty & 0 & 2 \\ 16 & 3 & 15 & \infty & 0 \\ 12 & 0 & 3 & 12 & \infty \end{pmatrix}$$

Row wise reduction sum = $10 + 2 + 2 + 3 + 4 = 21$

Apply column reduction method:

Deduct 1 (which is the minimum) from all values in the 1st column.

Deduct 3 (which is the minimum) from all values in the 2nd column.

The resulting row wise reduced cost matrix =

$$\begin{bmatrix} \infty & 10 & 10 & 0 & 1 \\ 12 & \infty & 11 & 2 & 0 \\ 0 & 3 & \infty & 0 & 2 \\ 15 & 3 & 12 & \infty & 0 \\ 11 & 0 & 0 & 12 & \infty \end{bmatrix}$$

→ [A]

Initial reduction cost = row red cost + col red cost
 $= 21 + 4$
 $= 25$

1 → 2
↓
row column

row 1, col 2, (2, 1) → ∞
must be set to ∞ in A matrix

$$\begin{bmatrix} \infty & \infty & \infty & \infty & \infty \\ \infty & \infty & 11 & 2 & 0 \\ 0 & \infty & \infty & 0 & 2 \\ 15 & \infty & 12 & \infty & 0 \\ 11 & \infty & 0 & 12 & \infty \end{bmatrix}$$

check whether each row and each col is having atleast one zero.

If not having atleast one zero we've ~~not~~ perform reduction process.

reduced cost = row red cost + col red cost
 $= 0 + 0 = 0$

cost(1,2) = initial reduced cost + reduced cost of (1,2) + A[1,2]
 $= 25 + 0 + 10$
 $= 35$

value of
↑
from
matrix
↓
reduced
matrix

cost(1,3)

1 → 3
↓
row col

row 1, col 3, (3,1) → ∞

$$\begin{bmatrix} \infty & \infty & \infty & \infty & \infty \\ 12 & \infty & \infty & 2 & 0 \\ \infty & 3 & \infty & 0 & 2 \\ 15 & 3 & \infty & \infty & 0 \\ 11 & 0 & \infty & 12 & \infty \end{bmatrix}$$

red cost = row red cost + col red cost
 $= 0 + 11$
 $= 11$

$$\begin{aligned} \text{cost}(1,3) &= 25 + 11 + A[1,3] \\ &= 25 + 11 + 17 \\ &= 53 \end{aligned}$$

1 → 4, row 1, col 4, (4,1) → ∞

$$\begin{bmatrix} \infty & \infty & \infty & \infty & \infty \\ 12 & \infty & 11 & \infty & 0 \\ 0 & 3 & \infty & \infty & 2 \\ \infty & 3 & 12 & \infty & 0 \\ 11 & 0 & 0 & \infty & \infty \end{bmatrix} \rightarrow B$$

red cost = row red cost + col red cost

$$\begin{aligned} &= 0 + 0 \\ &= 0 \end{aligned}$$

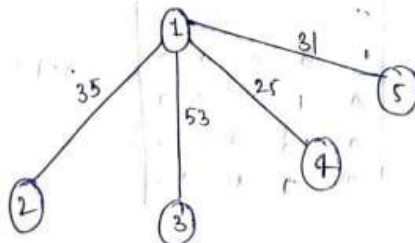
$$\begin{aligned} \text{cost}(1,4) &= 25 + 0 + A[1,4] \\ &= 25 + 0 + 0 \\ &= 25 \end{aligned}$$

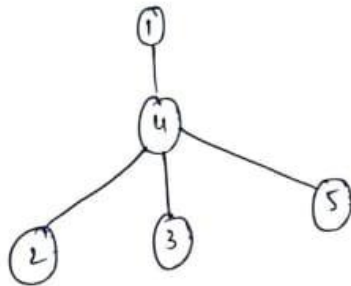
1 → 5, row 1, col 5, (5,1) → ∞

$$\begin{bmatrix} \infty & \infty & \infty & \infty & \infty \\ 12 & \infty & 11 & 2 & \infty \\ 0 & 3 & \infty & \infty & \infty \\ 15 & 3 & 12 & \infty & \infty \\ \infty & 0 & 0 & 12 & \infty \end{bmatrix} \begin{matrix} 2 \\ 3 \end{matrix} \Rightarrow \begin{bmatrix} \infty & \infty & \infty & \infty & \infty \\ 10 & \infty & 9 & 0 & \infty \\ 0 & 3 & \infty & 0 & \infty \\ 12 & 0 & 9 & \infty & \infty \\ \infty & 0 & 0 & 12 & \infty \end{bmatrix}$$

red cost = 5

$$\begin{aligned} \text{cost}(1,5) &= 25 + 5 + 1 \\ &= 31 \end{aligned}$$





(4,2) 4th row, 2nd col, (2,4) $\rightarrow \infty$

u $\rightarrow$ 2

$$\begin{bmatrix} \infty & \infty & \infty & \infty & \infty \\ \infty & \infty & 11 & \infty & 0 \\ 0 & \infty & \infty & \infty & 2 \\ \infty & \infty & \infty & \infty & \infty \\ 11 & \infty & 0 & \infty & \infty \end{bmatrix}$$

$\rightarrow$ (C)

$$\text{red cost} = 0 + 0 \\ = \underline{0}$$

(1,4) $\rightarrow$ 2

$$\begin{aligned} \text{cost}(4,2) &= 0 + 25 + \text{red cost of (1,4)} \\ &= 25 + 3 = 28 \end{aligned}$$

(4,3) 4th row, 3rd col, (3,4) $\rightarrow \infty$

$$\begin{bmatrix} \infty & \infty & \infty & \infty & \infty \\ 12 & \infty & \infty & \infty & 0 \\ \infty & 3 & \infty & \infty & 2 \\ \infty & \infty & \infty & \infty & \infty \\ 11 & 0 & \infty & \infty & \infty \end{bmatrix} \xrightarrow{2}$$

$$\begin{bmatrix} \infty & \infty & \infty & \infty & \infty \\ 12 & \infty & \infty & \infty & 0 \\ \infty & 1 & \infty & \infty & 0 \\ \infty & \infty & \infty & \infty & \infty \\ 11 & 0 & \infty & \infty & \infty \end{bmatrix}$$

$$\Rightarrow \begin{bmatrix} \infty & \infty & \infty & \infty & \infty \\ 1 & \infty & \infty & \infty & 0 \\ \infty & 1 & \infty & \infty & 0 \\ \infty & \infty & \infty & \infty & \infty \\ 0 & \infty & \infty & \infty & \infty \end{bmatrix}$$

$$\text{red cost} = 13$$

$$\begin{aligned} \text{cost}(4,3) &= 13 + 25 + \text{red cost of (4,3)} \\ &= 13 + 25 + 12 = 50 \end{aligned}$$

$$\begin{array}{r} 1 \\ 25 \\ 12 \\ \hline 50 \end{array}$$

$(4,5) \rightarrow 4^{\text{th}} \text{ row}, 5^{\text{th}} \text{ col}, (4,1) \rightarrow \infty$

$$\begin{bmatrix} \infty & \infty & \infty & \infty & \infty \\ 12 & \infty & 11 & \infty & \infty \\ 0 & 3 & \infty & \infty & \infty \\ \infty & \infty & \infty & \infty & \infty \\ 11 & 0 & 0 & \infty & \infty \end{bmatrix} \rightarrow \begin{bmatrix} \infty & \infty & \infty & \infty & \infty \\ 1 & \infty & 0 & \infty & \infty \\ 0 & 3 & \infty & \infty & \infty \\ \infty & \infty & \infty & \infty & \infty \\ 0 & 0 & 0 & \infty & \infty \end{bmatrix}$$

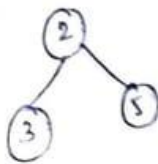
red = 11
cost

$$\text{cost}(4,5) = 11 + 25 + B[4,5]$$

$$= 36 + 0$$

$$= 36$$

(1,4,2)



(2,3)

2nd row, 3rd col, $(3,1) \rightarrow \infty$

$$\begin{bmatrix} \infty & \infty & \infty & \infty & \infty \\ \infty & \infty & \infty & \infty & \infty \\ \infty & \infty & \infty & \infty & 2 \\ \infty & \infty & \infty & \infty & \infty \\ 11 & \infty & \infty & \infty & 11 \end{bmatrix} \rightarrow \begin{bmatrix} \infty & \infty & \infty & \infty & \infty \\ \infty & \infty & \infty & \infty & \infty \\ \infty & \infty & \infty & \infty & 0 \\ \infty & \infty & \infty & \infty & \infty \\ 0 & \infty & \infty & \infty & \infty \end{bmatrix}$$

2

$$\rightarrow \begin{bmatrix} \infty & \infty & \infty & \infty & \infty \\ \infty & \infty & \infty & \infty & \infty \\ 0 & \infty & \infty & \infty & 0 \\ \infty & \infty & \infty & \infty & \infty \\ 0 & \infty & \infty & \infty & \infty \end{bmatrix}$$

red cost = 13

$$\text{cost}(2,3) = 13 + 28 + C[2,3]$$

$$= 13 + 28 + 11 = 52$$

$(2,5)$ 2nd row, 5th col $(5,1) \rightarrow \infty$

$$\begin{bmatrix} \infty & \infty & \infty & \infty & \infty \\ \infty & \infty & \infty & \infty & \infty \\ 0 & \infty & \infty & \infty & \infty \\ \infty & \infty & \infty & \infty & \infty \\ \infty & \infty & 0 & \infty & \infty \end{bmatrix} \rightarrow D$$

red cost = 0

$$\begin{aligned} \text{cost}(2,5) &= 0 + 28 + C[2,5] \\ &= 0 + 28 + 0 \\ &= 28 \end{aligned}$$

$(5,3)$ 5th row, 3rd col $(3,1) \rightarrow \infty$

$$\begin{bmatrix} \infty & \infty & \infty & \infty & \infty \\ \infty & \infty & \infty & \infty & \infty \\ 0 & \infty & \infty & \infty & \infty \\ \infty & \infty & \infty & \infty & \infty \\ \infty & \infty & \infty & \infty & \infty \end{bmatrix}$$

red cost = 0

$$\begin{aligned} \text{cost}(5,3) &= 0 + 28 + D[5,3] \\ &= 0 + 28 + 0 \\ &= 28 \end{aligned}$$

$1 \rightarrow 4 \rightarrow 2 \rightarrow 5 \rightarrow 3 \rightarrow 1$ with least cost as 28